Explain Your Framework

The walkthrough question that decides your level

PLAYWRIGHT · 2026 EDITION

Sreenidhi Rajakrishnan

Explain Your Framework: The Walkthrough

Playwright with TypeScript, 2026

1. Why this one question decides your level

Almost every senior QA and SDET interview opens with "walk me through your automation framework". It sounds like a warm up. It is not. Here is what they are measuring while you talk.

Did you build it or only use it. People who wrote test cases inside someone else's framework describe features. People who shaped a framework describe decisions. The moment you say "we log in once in a setup project, save the storage state to a file, and every worker reuses it so no test logs in through the UI", you have moved from user to owner.

Your vocabulary depth. If you say "we have a Playwright framework in TypeScript" and stop, they assume you know the tool, not the discipline. If you naturally reach for fixture, project, storage state, web first assertion, trace on first retry, sharding, page object, they know you have lived inside it.

How deep they can go. The interviewer has follow up questions ranked easy to hard. Your walkthrough tells them where to start. A vague answer means they open with "what is a locator" and you never reach the good material.

Communication, which is most of a senior job. A lead spends more time explaining systems than writing code. This question is a direct test of that.

Ownership language. "We had" is fine. "I decided", "I introduced", "I measured" is what gets you the band.

Many candidates have genuinely good experience and still answer badly, because they answer as a memory dump instead of a designed story. The fix is having a shape ready, and then a worked example underneath it that survives every follow up.

2. The sixty second answer in six beats

Sixty seconds is roughly 150 to 170 words spoken calmly. Six beats. Say them in order and you never ramble.

A note on the numbers. Every figure in the scripted lines below, and in the three versions in section 5, belongs to a worked example rather than to you. Swap in your own suite size, your own run times and your own pass rate before you say any of this out loud, because an interviewer only has to ask how you measured it. An honest small number always beats a borrowed big one.

Beat 1. Context, 8 seconds. What the product is, what you tested, scale. "It is a B2C insurance portal, web plus REST APIs, around 700 automated tests."

Beat 2. Stack, 8 seconds. Language, runner, structure, CI. "Playwright with TypeScript, the built in test runner, page objects and custom fixtures, running on GitHub Actions across sharded machines."

Beat 3. Structure, 15 seconds. The layout in one breath, and the rule behind it. "The repo splits into tests, pages, fixtures and a single config. Tests hold only business steps and assertions, pages hold locators and the actions for one screen, fixtures inject things like an authenticated page so no test logs in through

the UI."

Beat 4. One design decision you are proud of, 15 seconds. This is the beat that separates you. "The decision I am happiest with is authenticating once in a setup project and saving the storage state, then handing every test a signed in context through a fixture. That took about four minutes of repeated UI logins out of every run and removed a whole class of login flakiness."

Beat 5. Execution and reporting, 10 seconds. "Smoke runs on every pull request in about four minutes, full regression on merge across three shards in about nine, with the HTML report and a trace on first retry published as an artifact."

Beat 6. Handover line, 4 seconds. Invite the follow up and steer it to your strongest area. "Happy to go deeper into any part. The fixture design and the network stubbing are probably the most interesting bits."

That last line is deliberate. You are choosing the ground the deep dive happens on.

3. The complete worked example

Everything below is one coherent framework. If you can describe this and answer questions on any file in it, you will clear the walkthrough at mid to senior level.

3.1 Folder structure

```
automation/
  tests/
    auth/
      login.setup.ts    (the setup project, inside testDir on purpose)
    ui/
      login/
        login.spec.ts
        login-negative.spec.ts
      quote/
        quote-creation.spec.ts
        quote-premium.spec.ts
      policy/
        policy-purchase.spec.ts
    api/
      quote-api.spec.ts
      payment-api.spec.ts
  pages/
    login.page.ts
    quote.page.ts
    payment.page.ts
    components/
      header.component.ts
      cart.component.ts
  fixtures/
    index.ts
    auth.fixture.ts
    data.fixture.ts
  data/
    factories/
      applicant.factory.ts
    payloads/
      create-quote.json
  utils/
    api-client.ts
    env.ts
  playwright/
    .auth/
      user.json         (git ignored, holds the storage state)
  playwright.config.ts
  package.json
  tsconfig.json
  .github/workflows/e2e.yml
```

What lives where, and why.

`tests/` Only business readable steps and assertions. If someone from the business reads a file here they should follow what is being tested. No CSS selectors, no `waitForTimeout`, no raw API plumbing. The folders mirror the application's feature areas, not the technology.

`tests/auth/` The one time login that produces the storage state file. It sits inside `tests` deliberately, because a setup project only ever collects files under `testDir`. Park the setup file outside `testDir` and the project matches nothing, reports zero tests, writes no state, and every dependent project then fails on a file that was never created. This is the single most common way a copied Playwright config breaks on a clean checkout, and being able to say why is worth a lot in interview.

`pages/` One page object per screen. The class holds the locators and the actions for that page, and nothing about test logic. `components/` holds pieces that appear on many pages, like the header or the cart, so a page object composes components rather than duplicating them.

`fixtures/` The heart of a Playwright framework, and where most candidates are thin. Custom fixtures inject exactly what a test needs: a signed in `page`, a ready made page object, a seeded applicant. `index.ts` re exports a single `test` and `expect` that every spec imports.

`data/` Factories that generate unique data per run, and static JSON payloads for API calls. Nothing secret, ever.

`utils/` A thin API client used for seeding and teardown, and an `env.ts` that reads configuration once.

`playwright/.auth/` The storage state file the setup project writes, git ignored. Reports and state never get committed.

`playwright.config.ts` The single source of truth for base URL, timeouts, projects, retries, reporters and trace policy.

3.2 Naming conventions

Thing	Convention	Example
Spec file	lower kebab case, feature first	`quote-premium.spec.ts`
Page object file	lower kebab, `.page.ts`	`login.page.ts`
Component file	lower kebab, `.component.ts`	`header.component.ts`
Fixture file	lower kebab, `.fixture.ts`	`auth.fixture.ts`
Class name	PascalCase, matches the file	`LoginPage`
Test title	full sentence, business language	`user sees an error when the account is locked`
Method on a page	verb first, camelCase	`submitQuoteForm()`
Locator property	noun, camelCase, private	`private readonly emailInput`
Test id attribute	kebab, namespaced	`data-testid="login-submit"`

Two conventions worth calling out in interview.

Keep locators as private readonly properties resolved with the built in `getBy` methods, not as loose strings scattered through the test. The page object is the one place a selector lives, so a front end rename is a one line change.

Name test titles so the failure line alone tells you what broke. `test 3` tells nobody anything. `login fails with a locked account message after five wrong attempts` needs no investigation to triage.

3.3 A page object

`pages/login.page.ts`

```
import { type Page, type Locator, expect } from '@playwright/test';

export class LoginPage {
  private readonly emailInput: Locator;
  private readonly passwordInput: Locator;
  private readonly submitButton: Locator;
  private readonly errorBanner: Locator;

  constructor(private readonly page: Page) {
    this.emailInput = page.getByLabel('Email');
    this.passwordInput = page.getByLabel('Password');
    this.submitButton = page.getByRole('button', { name: 'Sign in' });
    this.errorBanner = page.getByRole('alert');
  }

  async open(): Promise<void> {
    await this.page.goto('/login');
    await expect(this.emailInput).toBeVisible();
  }

  async loginAs(email: string, password: string): Promise<void> {
    await this.open();
    await this.emailInput.fill(email);
    await this.passwordInput.fill(password);
    await this.submitButton.click();
  }

  async expectError(message: string): Promise<void> {
    await expect(this.errorBanner).toHaveText(message);
  }
}
```

The test that uses it, `tests/ui/login/login-negative.spec.ts`

```
import { test } from '../../../fixtures';

test.describe('login errors', () => {
  test('wrong password shows an error', async ({ loginPage }) => {
    await loginPage.loginAs('valid.user@example.com', 'WrongPassword1');
    await loginPage.expectError('Your email or password is incorrect');
  });

  test('locked account shows a support message', async ({ loginPage }) => {
    await loginPage.loginAs('locked.user@example.com', 'Correct123');
    await loginPage.expectError('Your account is locked, please contact support');
  });
});
```

Notice what the test file does NOT contain: no selectors, no waits, no page construction. It reads as behaviour. That is the point, and it is what the interviewer is looking for.

The three rules, say these out loud:

1. Tests never see selectors. Those live only inside the page object.
2. A page object waits for its own precondition. `open()` is not finished until the page is usable, so no caller ever adds a wait.
3. Cross page flows belong in a fixture or a flow helper, not inside a single page object calling another.

3.4 A custom fixture

This is the beat that separates a Playwright answer from a generic one. Most candidates can describe a page object. Far fewer can explain a fixture and why it matters.

`fixtures/index.ts`

```
import { test as base } from '@playwright/test';
import { LoginPage } from '../pages/login.page';
import { QuotePage } from '../pages/quote.page';
import { ApiClient } from '../utils/api-client';

type Fixtures = {
  loginPage: LoginPage;
  quotePage: QuotePage;
  api: ApiClient;
};

export const test = base.extend<Fixtures>({
  // A page object, built fresh per test, injected by name
  loginPage: async ({ page }, use) => {
    await use(new LoginPage(page));
  },

  quotePage: async ({ page }, use) => {
    await use(new QuotePage(page));
  },

  // A thin API client for seeding data. The `request` fixture is its own
  // isolated APIRequestContext, so it does not carry the browser's cookies.
  // Auth is supplied explicitly, through storageState or a token header.
  api: async ({ request }, use) => {
    await use(new ApiClient(request));
  },
});

export { expect } from '@playwright/test';
```

Because each browser project points `storageState` at the file the setup project wrote, every `page` a test receives is already signed in. No test spends four seconds logging in through the UI, and login flakiness disappears from the whole suite. A test that genuinely needs the login screen simply overrides the state:

```
test.use({ storageState: { cookies: [], origins: [] } });
```

Points to make about your fixtures:

- A fixture only runs if a test names it, so unused setup costs nothing.
- Fixtures can depend on each other. `quotePage` could depend on `api` to seed a draft quote before the test starts.
- A worker scoped fixture, `{ scope: 'worker' }`, is for something expensive shared across a worker, like a database pool. A per test page must stay test scoped.
- Fixtures replace the scattered `beforeEach` blocks most Selenium era frameworks rely on, and they compose far more cleanly.
- The `request` fixture is a separate isolated API context, not a view of the browser. It does not pick up cookies from the signed in `page`. If a call genuinely has to run as the browser session, use `page.request` or `context.request`. Otherwise give the API client its own auth, a token header or the same saved storage state. Very few candidates know this, and getting it wrong is what makes a seeded API call come

back as a 401.

3.5 The config

`playwright.config.ts`

```
import { defineConfig, devices } from '@playwright/test';
import { env } from './utils/env';

// The setup project writes this file. The browser projects read it.
const STATE = 'playwright/.auth/user.json';

export default defineConfig({
  testDir: './tests',
  timeout: 30_000,
  expect: { timeout: 10_000 },
  fullyParallel: true,
  forbidOnly: !!process.env.CI,
  retries: process.env.CI ? 2 : 0,
  workers: process.env.CI ? 4 : undefined,
  // Blob on CI so the shards can be merged into one report afterwards.
  reporter: process.env.CI
    ? [['blob']]
    : [['html', { open: 'never' }], ['list']],

  // Nothing that a signed out project cannot live with goes in here.
  use: {
    baseURL: env.baseUrl,
    trace: 'on-first-retry',
    screenshot: 'only-on-failure',
    video: 'retain-on-failure',
  },

  projects: [
    // Runs first, signs in, writes the storage state the others reuse.
    // The file lives in tests/auth/, inside testDir, so this project
    // actually collects it.
    { name: 'setup', testMatch: /login\.setup\.ts/ },
    {
      name: 'chromium',
      use: { ...devices['Desktop Chrome'], storageState: STATE },
      dependencies: ['setup'],
    },
    {
      name: 'firefox',
      use: { ...devices['Desktop Firefox'], storageState: STATE },
      dependencies: ['setup'],
    },
  ],
});
```

`tests/auth/login.setup.ts`

```
import { test as setup, expect } from '@playwright/test';
import { env } from '../../utils/env';

const STATE = 'playwright/.auth/user.json';

setup('authenticate once', async ({ page }) => {
  await page.goto('/login');
  await page.getByLabel('Email').fill(env.user);
  await page.getByLabel('Password').fill(env.password);
  await page.getByRole('button', { name: 'Sign in' }).click();
  await expect(page.getByRole('heading', { name: 'Dashboard' })).toBeVisible();
  await page.context().storageState({ path: STATE });
});
```

Say these four things about the config, they all land well:

1. `storageState` sits on each dependent project, never in the root `use` block. Anything in the root `use` is inherited by every project including `setup`, so a root level `storageState` tells the sign in job to load a session file it has not written yet. On a clean checkout, where the state folder is git ignored and therefore empty, the run dies before a single test with `Error reading storage state from ...`. Moving that one line down into the browser projects is the difference between a config that runs anywhere and one that only runs on the machine that already has the file. Very few candidates can explain this, and it is a genuinely good answer.
2. Secrets never live in the repo. `env.ts` reads them from environment variables that CI injects from its secret store. A password in the config is a red flag and interviewers do check.
3. The environment is chosen at run time. One `BASE_URL` variable and the whole suite points at dev, QA or staging with no branch and no file to edit.
4. `trace: 'on-first-retry'` is the setting I care about most. It costs nothing on green runs and gives a full DOM, network and console recording on the exact flaky run, so I debug from evidence rather than guesses.

A quick way to prove the wiring is right before you ever demo it: `npx playwright test --list` prints what each project collected. If the setup project reports zero tests, your setup file is outside `testDir` and no state is being written, whatever the rest of the config says.

3.6 Data, network and the API client

The rule I follow: the UI test tests the UI, and everything else is set up through the API.

```
// utils/api-client.ts
import { type APIRequestContext, expect } from '@playwright/test';

export class ApiClient {
  constructor(private readonly request: APIRequestContext) {}

  async createApplicant(age: number): Promise<{ id: string }> {
    const res = await this.request.post('/api/applicants', {
      data: { email: `qa.${Date.now()}@mailtest.example.com`, age },
    });
    await expect(res).toBeOK();
    return res.json();
  }
}
```

Seeding a UI test through that client keeps the test about the one thing it checks:

```
test('applicant below eighteen is rejected', async ({ api, quotePage }) => {
  const applicant = await api.createApplicant(17);
  await quotePage.openForApplicant(applicant.id);
  await quotePage.expectRejection('Applicant must be at least 18 years old');
});
```

And for a test that must not depend on backend state at all, stub the network:

```
test('empty cart shows a friendly message', async ({ page }) => {
  await page.route('**/api/cart', route =>
    route.fulfill({ json: { items: [] } }));
  await page.goto('/cart');
  await expect(page.getByText('Your cart is empty')).toBeVisible();
});
```

3.7 CI integration

`.github/workflows/e2e.yml`

```
name: e2e
on:
  pull_request:
  push:
    branches: [main]

jobs:
  # Gate one. Every pull request, the tagged smoke set only, one browser,
  # under five minutes. Tests carry { tag: '@smoke' } in their title options.
  smoke:
    if: github.event_name == 'pull_request'
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: 20, cache: npm }
      - run: npm ci
      - run: npx playwright install --with-deps chromium
      - run: npx playwright test --project=chromium --grep @smoke --reporter=html
        env:
          BASE_URL: ${{ vars.QA_BASE_URL }}
          APP_USER: ${{ secrets.APP_USER }}
          APP_PASSWORD: ${{ secrets.APP_PASSWORD }}
      - uses: actions/upload-artifact@v4
        if: ${{ !cancelled() }}
        with:
          name: smoke-report
          path: playwright-report/
          retention-days: 1
```

Gate two, in the same file. Full regression on merge to main, across three machines.

```
regression:
  if: github.event_name == 'push'
  runs-on: ubuntu-latest
  strategy:
    fail-fast: false
    matrix:
      shard: [1, 2, 3]
  steps:
    - uses: actions/checkout@v4
    - uses: actions/setup-node@v4
      with: { node-version: 20, cache: npm }
    - run: npm ci
    # Install exactly the browsers the config declares as projects.
    - run: npx playwright install --with-deps chromium firefox
    - run: npx playwright test --shard=${{ matrix.shard }}/3
      env:
        BASE_URL: ${{ vars.QA_BASE_URL }}
        APP_USER: ${{ secrets.APP_USER }}
        APP_PASSWORD: ${{ secrets.APP_PASSWORD }}
    - uses: actions/upload-artifact@v4
      if: ${{ !cancelled() }}
      with:
        name: blob-report-${{ matrix.shard }}
        path: blob-report/
        retention-days: 1
```

And the job that makes sharding readable. Three shards produce three separate blob reports. Without this job you read three sets of results and guess which failure matters.

```
merge-reports:
  needs: [regression]
  if: ${{ !cancelled() && github.event_name == 'push' }}
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v4
    - uses: actions/setup-node@v4
      with: { node-version: 20, cache: npm }
    - run: npm ci
    - uses: actions/download-artifact@v4
      with:
        path: all-blob-reports
        pattern: blob-report-*
        merge-multiple: true
    - run: npx playwright merge-reports --reporter html ./all-blob-reports
    - uses: actions/upload-artifact@v4
      with:
        name: html-report
        path: playwright-report/
```

CI points that make you sound senior:

- Two gates, not one, and they are two separate jobs picked by `github.event_name`. A tagged smoke set on every pull request under five minutes, full regression across shards on merge to main. A slow gate gets bypassed, and then you have no gate.
- Sharding with a matrix runs the suite across three machines in parallel. Each shard writes a blob report, and a final job runs `playwright merge-reports` to fold them into one HTML report. Say the merge step out loud, because sharding without it just means three partial reports and nobody knowing which failure matters.

- The browsers installed in CI match the projects the config declares. Declaring a firefox project and installing only chromium is a five second fix and a broken pipeline, and it is worth naming as the sort of drift you check for.
- Secrets come from the CI secret store, never the repo. The base URL is a plain variable, so the same run points anywhere.
- The report and trace upload on failure, so anyone can debug without your laptop. If it only reproduces on your machine, nobody can help.
- The suite runs locally with the exact same command. If it only works in CI, it cannot be debugged.

4. The follow up questions they always ask next

Two of these come straight out of the config and the fixtures you have just described, so they are the ones where a vague answer costs you most. The broader strategy questions that usually follow, test data, flaky tests, what to automate, what to leave manual, are answered at length in the Real-Time Scenario Q&A companion, so use that for those and keep this section about your framework.

"How do you run against multiple environments?"

One variable. `BASE_URL` chooses dev, QA or staging and everything follows, because `env.ts` reads it once and the config uses it. No branch per environment, no config file to edit, no environment name hard coded in a test. Secrets are not in the repo, they come from environment variables that CI injects from its secret store. The `env.ts` reader throws immediately on a missing variable, so a misconfiguration fails in two seconds rather than running the whole suite against nothing.

"Why fixtures over the beforeEach hooks you would use elsewhere?"

Because fixtures are lazy, composable and typed. A `beforeEach` runs for every test in the block whether the test needs it or not. A fixture runs only when a test names it, so a test that does not touch the API never pays for the API client. Fixtures depend on each other cleanly, so a seeded quote fixture can depend on an API fixture, and TypeScript checks the whole graph. And teardown lives in the same function after the `use` call, so setup and cleanup sit together rather than in two separate hooks that drift apart.

The readability half matters just as much. When I open a spec I can read what a test depends on straight off the first line of its signature. With `beforeEach` I have to scroll up and hold shared state in my head, and that state may be set three describe blocks higher. Having the dependency written into the test signature is what keeps a large suite changeable.

5. Three sized versions of the walkthrough

Version A. For 1 to 3 years

The goal is not to sound like an architect. It is to sound like someone who understands the framework they work in. Overclaiming is the main way people fail at this level.

> "Sure. We test a B2C insurance portal, web and APIs, and the suite is around 700 Playwright tests in TypeScript.

> The structure is that tests hold the business steps and assertions, pages hold the locators and actions for each screen with one page object per page, and fixtures inject things like a page object or a signed in page so the tests stay short. There is a data folder with factories that generate unique data, and the config holds the base URL and timeouts in one place.
>
> My work has mostly been in the quote and login areas. I have written around a hundred tests there, and I built the quote page object from scratch when that feature was added. I also moved a set of similar premium tests onto a parameterised loop so one data change did not mean editing five tests.
>
> Everything runs on GitHub Actions, smoke on every pull request and full regression on merge. I do not own the pipeline but I know how to read the HTML report and open a trace, and I triage failures for my area in the morning.
>
> One thing I learned the hard way is test data. My early tests reused a fixed user and they kept failing once we went parallel. Now every test creates its own data through the API, and that fixed it.
>
> Happy to go into any part in more detail."

Why it works: clear structure, one real contribution, one honest lesson, no pretending to own the pipeline. That last bit builds trust, and trust is what gets a junior hired.

Version B. For 3 to 8 years

Design decisions with reasons and outcomes. This is the version most people will use.

> "It is a B2C insurance portal, web plus REST APIs, around 700 Playwright tests in TypeScript on the built in runner, running on GitHub Actions.
>
> The repo splits into tests, pages, fixtures, data and a single config. Tests contain only business steps and web first assertions, no selectors, no waits. Pages hold one page object per screen with locators as private properties, and shared pieces like the header live as components a page composes. Fixtures are where most of the design is: they inject page objects and a signed in page, and the interesting one authenticates once in a setup project, saves the storage state, and hands every test a signed in context.
>
> Three decisions I would call out. First, that storage state fixture, which took repeated UI logins out of every test and removed login flakiness entirely, and saved about four minutes a run. Second, test data independence, every test creating its own data through the API, which is what let us run fully parallel and killed the failures we used to get after a database refresh. Third, trace on first retry, so a flaky run leaves a full recording and I debug from evidence rather than reruns.
>
> On execution, smoke runs on every pull request in about four minutes, and full regression runs on merge across three shards in about nine, with retries that mark a rerun pass as flaky rather than green so flakiness stays visible. The HTML report and trace upload as artifacts.
>
> Automation code goes through the same review bar as product code, with lint and a type check in the pipeline before the tests run.
>
> Happy to go deeper anywhere. The fixture design and the network stubbing are probably the most interesting parts."

Version C. For lead and architect level

At this level they care less about folder structure and more about strategy, cost, people and how you decide.

> "I will give you the shape, then the reasoning, and please stop me wherever you want detail.
>
> The context is a B2C insurance platform, four feature teams, web and API. When I took it over there were two automation efforts, roughly 1100 tests, a Selenium suite taking over three hours that people had quietly stopped trusting, and a pass rate around 84 percent where nobody could tell you if a red build was a real bug.
>
> My first move was not technical. I spent two weeks measuring: which tests failed and then passed on rerun, which had never caught a real defect, and which business flows had no coverage. That data was what got me buy in, because I could show that a few hundred tests were producing noise and several revenue critical flows had no automation.
>
> Then consolidation onto Playwright and TypeScript. One repo, one structure. Page objects with locators encapsulated, fixtures for setup, storage state reused so nothing logs in through the UI, environment chosen by one variable, secrets from the CI store.
>
> The three strategic calls I made. One, delete before you add. We removed a couple of hundred tests that duplicated each other or tested nothing, which is culturally hard, so I made it a data conversation not an opinion one. Two, test data independence as non negotiable, the enabler for parallel execution and for running after a database refresh without a Monday firefight. Three, treat the automation repo as a product with an owner, a backlog, a review bar, lint and types in CI, and a README that gets a new joiner to a green local run in under thirty minutes.
>
> On tooling I did not do a big bang rewrite. New work went onto Playwright, the legacy Selenium suite kept running and we ported it feature by feature as we touched it, because a rewrite with no new coverage is very hard to justify and stalls halfway.
>
> Outcomes after two quarters: regression from three hours plus to under ten minutes with sharding, flaky rate from around ten percent to under two, escaped defects in covered areas down from an average of five a release to one. The number I report to leadership is not test count, it is which business flows are protected and how quickly we know something broke.
>
> The part I would do differently is that I under invested in API level testing for the first quarter, so too much value sat in slow UI tests. If I did it again I would push coverage down the pyramid earlier.
>
> Where would you like me to go deeper?"

Why it works at lead level: it opens with a problem and data, not folders. It includes a people decision, a deliberate non decision (no big bang rewrite), measurable outcomes, and one honest regret. That regret line is disproportionately powerful, because it signals someone who reflects rather than defends.

6. What not to say

"We follow the hybrid framework, page object plus data driven plus BDD." Repeated so often in Indian interviews that it now signals a memorised answer. Replace it with the actual structure and one real decision.

"We use POM and that is our framework." Naming a pattern is not explaining a framework. In Playwright the more interesting design is in the fixtures. If page object is all you say, the follow up will be about fixtures and you want to be the one who raised them.

"I don't know the folder structure, someone else set it up." Fatal, because it means you never looked. If you did not build it, describe it accurately and say what you contributed.

"We automate everything" or "we have 100 percent coverage." No team does, and claiming it makes everything else suspect.

"We add a wait when the app is slow." Instant credibility loss in a Playwright interview, because auto waiting exists precisely so you do not. If you genuinely have a few `waitForTimeout` calls, say "we have a couple of legacy sleeps I have been replacing with web first assertions", which is honest and shows direction.

"Flaky tests, we just retry them." Says you tolerate noise. Replace with measure, read the trace, fix the cause, and keep the retry only as a visible flag, not a cure.

"We don't review test code." Tells them your automation is unreviewed, which usually means duplicated and fragile.

Reading out your entire folder tree file by file. Sixty seconds, not five minutes. The tree is on this page for you, not for them.

Saying "we" for everything. The interviewer cannot hire the team. Use "I" for what you did and "we" for team context.

Naming your client or their internal systems. Say "a large insurance client" and move on. Discretion about a previous employer is read as a positive, because they know you will be discreet about them too.

7. The one line to leave them with

When the deep dive winds down, hand it back cleanly and specifically, not with a shrug. Something like:

> "So in one line: business readable tests, all the real design in the page objects and fixtures, data every test owns itself, and a trace on every flaky run so nothing is a mystery. The fixture and storage state work is what I am proudest of, and the network stubbing is where I would take a new joiner first."

That sentence does three things. It proves you can summarise a system, it repeats your strongest ground one last time, and it leaves the interviewer with a clean note to write down. The walkthrough is not a memory test. It is a demonstration that you can explain a system calmly, and that is most of what a senior role is.